

Week 5

Task 1

Aim: To find the maximum possible sum of a subarray in a circular array, considering both normal and wrap-around subarrays.

Algorithm:

1. Use Kadane's algorithm to find the maximum subarray sum.
2. Use Kadane's algorithm to find the minimum subarray sum.
3. Compute the total sum of the array.
4. Calculate circular sum as $\text{totalSum} - \text{minSubarraySum}$.
5. If all elements are negative, return the normal maximum subarray sum.
6. Otherwise, return the maximum of normal sum and circular sum.

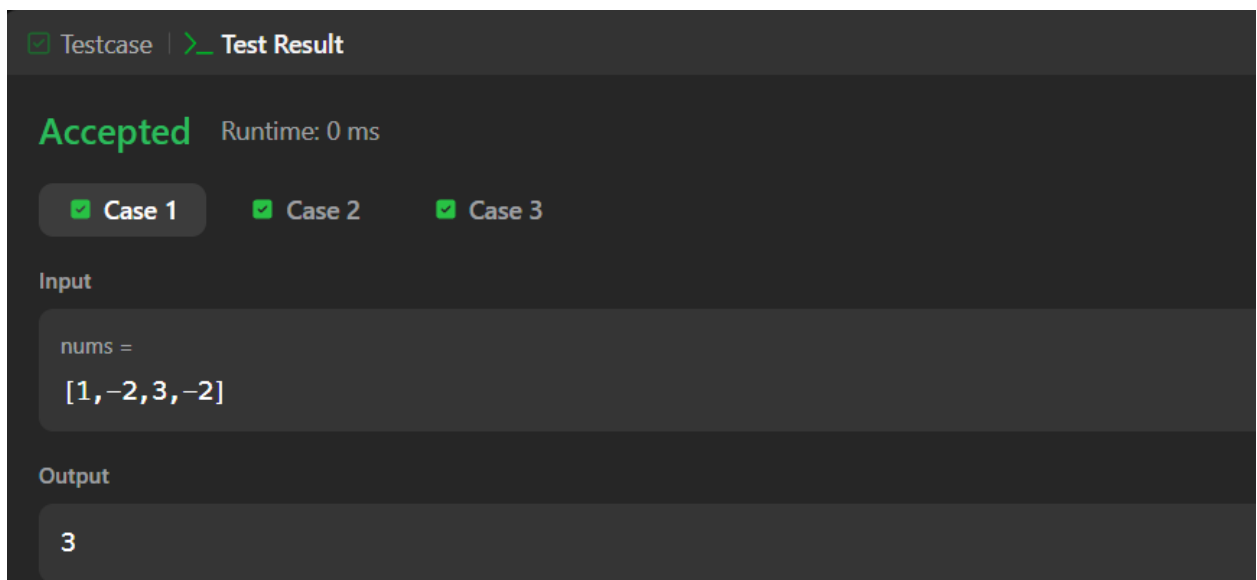
Program:

```
class Solution {  
  
    public int maxSubarraySumCircular(int[] nums) {  
  
        int total = 0;  
  
        int maxSum = nums[0];  
  
        int curMax = 0;  
  
        int minSum = nums[0];  
  
        int curMin = 0;  
  
        for (int n : nums) {  
  
            curMax = Math.max(curMax + n, n);  
  
            maxSum = Math.max(maxSum, curMax);  
  
            curMin = Math.min(curMin + n, n);  
  
            minSum = Math.min(minSum, curMin);  
  
  
            total += n;  
  
        }  
    }  
}
```

```
    if (maxSum < 0) return maxSum;

    return Math.max(maxSum, total - minSum);
}
}
```

Output:



The screenshot displays a test result interface with a dark theme. At the top, there are two tabs: 'Testcase' (checked) and 'Test Result'. Below the tabs, the word 'Accepted' is shown in green, followed by 'Runtime: 0 ms'. Underneath, there are three buttons, each with a green checkmark and the text 'Case 1', 'Case 2', and 'Case 3' respectively. Below these buttons is a section labeled 'Input' which contains the text 'nums =' followed by '[1,-2,3,-2]'. At the bottom, there is a section labeled 'Output' which contains the number '3'.

Result: To find the maximum possible sum of a subarray in a circular array, considering both normal and wrap-around subarrays has successfully verified.

Task 2

Aim: To find the number of contiguous segments of length m in the chocolate bar such that the sum of elements equals d .

Algorithm:

1. Initialize $\text{count} = 0$ and $\text{sum} = 0$.
2. Traverse the list from index 0 to $n-1$.
3. Add the current element to sum .
4. If window size exceeds m , subtract the element leaving the window ($i - m$).
5. When window size becomes m , check:
 - a. If $\text{sum} == d$, increment count .
6. After traversal, return count

Program:

```
class Result {  
  
    public static int birthday(List<Integer> s, int d, int m) {  
  
        int count = 0;  
  
        int sum = 0;  
  
        for (int i = 0; i < s.size(); i++) {  
  
            sum += s.get(i);  
  
            if (i >= m) {  
  
                sum -= s.get(i - m);  
  
            }  
  
            if (i >= m - 1 && sum == d) {  
  
                count++;  
  
            }  
  
        }  
  
        return count;  
  
    }  
}
```

Output:

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

✔ Sample Test case 0

✔ Sample Test case 1

✔ Sample Test case 2

Input (stdin)

```
1 5
2 1 2 1 3 2
3 3 2
```

[Download](#)

Your Output (stdout)

```
1 2
```

Expected Output

```
1 2
```

[Download](#)

Result: To find the number of contiguous segments of length m in the chocolate bar such that the sum of elements equals d has successfully executed .

Task 3

Aim: To compute the maximum sum of a contiguous subarray and the maximum sum of a non-contiguous subsequence from a given integer array.

Algorithm:

1. Initialize currentSum and maxSubarray with the first element.
2. Traverse the array:
3. Update currentSum = max(arr[i], currentSum + arr[i]).
4. Update maxSubarray with the maximum value.
5. For subsequence:
6. Add all positive elements to maxSubsequence.
7. Track the maximum element in the array.
8. If no positive elements exist, set subsequence sum to the maximum element.
9. Return both sums.

Program:

```
public static List<Integer> maxSubarray(List<Integer> arr) {  
  
    int maxSubarray = arr.get(0);  
  
    int currentSum = arr.get(0);  
  
    int maxSubsequence = 0;  
  
    int maxElement = arr.get(0);  
  
    for (int i = 0; i < arr.size(); i++) {  
  
        int val = arr.get(i);  
  
        if (i > 0) {  
  
            currentSum = Math.max(val, currentSum + val);  
  
            maxSubarray = Math.max(maxSubarray, currentSum);  
  
        } if (val > 0) {  
  
            maxSubsequence += val;  
  
        }  
  
        maxElement = Math.max(maxElement, val);  
  
    }  
}
```

```

if (maxSubsequence == 0) {

    maxSubsequence = maxElement;

} return Arrays.asList(maxSubarray, maxSubsequence);

}

```

Output:

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

✓ **Sample Test case 0**

✓ Sample Test case 1

Input (stdin)

1	2
2	4
3	1 2 3 4
4	6
5	2 -1 2 3 4 -5

Your Output (stdout)

1	10 10
2	10 11

Expected Output

1	10 10
---	-------

Result: To compute the maximum sum of a contiguous subarray and the maximum sum of a non-contiguous subsequence from a given integer array was successfully executed.

Task 4

Aim: To find the maximum sum of a contiguous subarray from the given integer array.

Algorithm:

1. Initialize currentSum and maxSum with the first element.
2. Traverse the array from the second element.
3. For each element, update currentSum as the maximum of the current element or the sum of current element with previous sum.
4. Update maxSum if currentSum is greater.
5. Return maxSum.

Program:

```
class Solution {  
  
    public int maxSubArray(int[] nums) {  
  
        int currentSum = nums[0];  
  
        int maxSum = nums[0];  
  
        for (int i = 1; i < nums.length; i++) {  
  
            if (currentSum + nums[i] > nums[i]) {  
  
                currentSum = currentSum + nums[i];  
  
            } else {  
  
                currentSum = nums[i];  
  
            }  
  
            if (currentSum > maxSum) {  
  
                maxSum = currentSum;  
  
            }  
  
        }  
  
        return maxSum;  
  
    }  
}
```

Output:

☒ Testcase | [> Test Result](#)

Accepted Runtime: 0 ms

☒ Case 1 ☒ Case 2 ☒ Case 3

Input

nums =
[-2,1,-3,4,-1,2,1,-5,4]

Output

6

Expected

6

Result: To find the maximum sum of a contiguous subarray from the given integer array has successfully verified.

Task 5

Aim: Convert a given string into a 32-bit signed integer by handling spaces, sign, digits, and overflow.

Algorithm:

1. Remove leading spaces using trim().
2. If string is empty, return 0.
3. Initialize variables
4. sign = 1
5. i = 0
6. result = 0
7. Check sign
8. If '+' → positive
9. If '-' → negative
10. Move index forward.
11. Traverse digits
12. While character is digit, convert and build number
13. result = result * 10 + digit
14. Check overflow
15. If result exceeds integer range, return Integer.MAX_VALUE or Integer.MIN_VALUE.
16. Return final answer
17. result * sign

Program:

```
class Solution {  
  
    public int myAtoi(String s) {  
  
        s = s.trim();  
  
        if (s.length() == 0) return 0;  
  
        int sign = 1;  
  
        int i = 0;  
  
        long result = 0;  
  
        if (s.charAt(i) == '-' || s.charAt(i) == '+') {  
  
            if (s.charAt(i) == '-') sign = -1;  
  
            i++;  
  
        }  
    }  
}
```

```

while (i < s.length() && Character.isDigit(s.charAt(i))) {

    result = result * 10 + (s.charAt(i) - '0');

    if (sign * result > Integer.MAX_VALUE) return Integer.MAX_VALUE;

    if (sign * result < Integer.MIN_VALUE) return Integer.MIN_VALUE;

    i++;

}

return (int)(sign * result);

}

}

```

Output:

The screenshot shows a test result interface with a dark theme. At the top, there are two tabs: 'Testcase' and 'Test Result', with 'Test Result' being the active tab. Below the tabs, the word 'Accepted' is displayed in green, followed by 'Runtime: 0 ms'. A row of five test cases is shown, each with a green checkmark and the label 'Case 1' through 'Case 5'. Below this, there are three sections: 'Input', 'Output', and 'Expected'. The 'Input' section shows 's =' followed by a text box containing '"42"'. The 'Output' section shows a text box containing '42'. The 'Expected' section shows a text box containing '42'. At the bottom of the interface, there is a link that says 'Contribute a testcase' with a heart icon.

Result: Convert a given string into a 32-bit signed integer by handling spaces, sign, digits, and overflow has successfully executed.

Task 6

Aim: To find the minimum number of deletions required so that no two adjacent characters in the string are the same.

Algorithm:

1. Initialize a variable `count = 0`.
2. Traverse the string from index 1 to end.
3. Compare current character with previous character.
4. If both characters are the same, increment `count` (delete one character).
5. After traversal, return `count`.

Program:

```
public static int alternatingCharacters(String s) {  
  
    int count = 0;  
  
    for (int i = 1; i < s.length(); i++) {  
  
        if (s.charAt(i) == s.charAt(i - 1)) {  
  
            count++;  
  
        }  
  
    }  
  
    return count;  
  
}
```

Output:

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

✓ Sample Test case 0

✓ Sample Test case 1

✓ Sample Test case 2

Input (stdin)

```
1 5
2 AAAA
3 BBBB
4 ABABABAB
5 BABABA
6 AAABBB
```

Your Output (stdout)

```
1 3
2 4
3 0
4 0
```

Result: To find the minimum number of deletions required so that no two adjacent characters in the string are the same has successfully executed.